

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

Facultad de Ingeniería

Escuela Profesional de Ingeniería en Informática y Sistemas

HELA

Especificación de Arquitectura de Software (SAD)

Sistema web de gestión para restaurantes

Línea base tecnológica vigente · Versión 2.0

Ingeniería de Software I · 2026-II

Equipo NEXO

- Oscar Fabian Choque Palza, 2023-119010
- Alex Jeanpier Chata Chino, 2024-119012
- Helber Junior Lope Apomayta, 2021-119119,
- Elvis Ronald Arocutipa Quispe, 2024-119056

Tacna, Perú · 2026

Control documental

Tabla 1

Control documental

Campo	Valor
Documento	HELA - Especificación de Arquitectura de Software (SAD)
Versión	2.0
Estado	Línea base tecnológica vigente
Curso	Ingeniería de Software I
Periodo	2026-II
Institución	Universidad Nacional Jorge Basadre Grohmann
Equipo	Nexo
Repositorio oficial	https://github.com/Alex-Jeanpier-Chata/is-2026-nexo

Tabla 2

Historial de versiones

Versión	Fecha	Descripción
1.0	28/08/2026	Línea base arquitectónica documentada.
2.0	01/09/2026	Línea base tecnológica vigente; actualización de vistas arquitectónicas, persistencia, componentes y despliegue.

Resumen ejecutivo

HELA es un sistema web SaaS orientado a la gestión operativa, comercial y administrativa de restaurantes. La arquitectura organiza la solución como un monolito modular, con un frontend construido con HTML5, CSS3 y JavaScript Vanilla, una API REST implementada con Node.js y Express.js y una base de datos relacional MySQL.

El frontend se comunica con la API mediante HTTP/JSON utilizando Fetch API. El backend agrupa las funciones por módulos del dominio y mantiene una separación sencilla entre rutas, controladores, servicios y acceso a datos. La persistencia se realiza mediante MySQL utilizando un driver compatible con Node.js, con mysql2 como referencia de implementación.

El contrato hela-openapi.yaml permanece independiente de la tecnología de implementación y continúa siendo la fuente de verdad para rutas, parámetros, solicitudes, respuestas, errores y trazabilidad con los casos de uso. La arquitectura conserva el aislamiento de información por restaurante y sucursal, el control de acceso, la auditoría y las integraciones de facturación electrónica e impresión sustentadas por la documentación funcional de HELA.

El entorno inicial de construcción y validación es local. La API se expone en <http://localhost:3000>, de acuerdo con el servidor declarado en el contrato OpenAPI vigente. La publicación en infraestructura externa se mantiene como una decisión posterior, sin alterar la arquitectura lógica del sistema.

Palabras clave: arquitectura de software, C4, Node.js, Express.js, MySQL, OpenAPI, monolito modular, HELA.

Índice general

Resumen ejecutivo	2
Control documental	2
1. Propósito, alcance y fuentes	6
1.1 Propósito.....	6
1.2 Alcance.....	6
1.3 Fuentes documentales.....	6
2. Drivers arquitectónicos	6
2.1 Drivers funcionales.....	7
2.2 Drivers de calidad.....	7
2.3 Restricciones arquitectónicas.....	7
3. Arquitectura de solución	8
3.1 Estilo arquitectónico.....	8
3.2 Principios de diseño.....	8
3.3 Línea base tecnológica.....	8
3.4 Flujo general de la solución.....	9
4. Vistas arquitectónicas C4	9
4.1 C4 Nivel 1 - Contexto.....	9
4.2 C4 Nivel 2 - Contenedores.....	10
4.3 C4 Nivel 3 - Componentes de la API HELA.....	11
5. Organización del código	11
5.1 Estructura general.....	11
5.2 Organización de un módulo backend.....	12
5.3 Criterios de organización.....	12
6. Persistencia y aislamiento de información	13
6.1 Persistencia relacional.....	13
6.2 Acceso a datos.....	13
6.3 Aislamiento por restaurante y sucursal.....	13
6.4 Transacciones e integridad histórica.....	13
7. Seguridad	14
7.1 Autenticación.....	14
7.2 Autorización y contexto.....	14
7.3 Canal público QR.....	14
7.4 Credenciales y consultas.....	14
7.5 Auditoría.....	14
8. Integraciones externas	15
8.1 Facturación electrónica.....	15
8.2 Impresión.....	15
8.3 Archivos y recursos.....	15
8.4 Tratamiento de fallos.....	15
9. Despliegue	15
9.1 Entorno local inicial.....	15
9.2 Distribución local.....	15
9.3 Publicación externa.....	16
10. Flujos arquitectónicos representativos	16
10.1 Inicio de sesión.....	17
10.2 Pedido temporal QR y validación.....	17
10.3 Pedido oficial y comanda.....	17
10.4 Pago y documento comercial.....	17
10.5 Facturación electrónica.....	17
11. Contrato OpenAPI	17
11.1 Fuente de verdad HTTP.....	17
11.2 Independencia tecnológica.....	17
11.3 Cobertura contractual vigente.....	17
11.4 Reglas de implementación.....	18
12. Trazabilidad arquitectónica	18
12.1 Módulos y componentes.....	18

12.2 Casos de uso y contrato.....	18
12.3 Cadena de trazabilidad.....	19
13. Escenarios de calidad	19
14. Riesgos y decisiones pendientes	20
14.1 Riesgos principales	20
14.2 Decisiones pendientes.....	20
15. Artefactos vinculados	21
15.1 Ubicación oficial.....	21
15.2 Regla de referencia	21
16. Referencias	21

Índice de tablas

Tabla 1. Control documental.....	2
Tabla 2. Historial de versiones	2
Tabla 3. Fuentes documentales de la arquitectura	6
Tabla 4. Drivers arquitectónicos	7
Tabla 5. Línea base tecnológica.....	8
Tabla 6. Responsabilidades de los contenedores.....	10
Tabla 7. Reglas de persistencia y aislamiento	13
Tabla 8. Controles de seguridad.....	14
Tabla 9. Integraciones externas.....	15
Tabla 10. Componentes del despliegue local	15
Tabla 11. Flujos arquitectónicos representativos	16
Tabla 12. Trazabilidad módulo-componente	18
Tabla 13. Trazabilidad OpenAPI por etiqueta	18
Tabla 14. Escenarios de calidad	19
Tabla 15. Decisiones pendientes	20
Tabla 16. Registro oficial de artefactos	21

Índice de figuras

Figura 1. C4 Nivel 1 - Contexto de HELA.....	9
Figura 2. C4 Nivel 2 - Contenedores de HELA	10
Figura 3. C4 Nivel 3 - Componentes de la API HELA	11
Figura 4. Despliegue local inicial de HELA	16

1. Propósito, alcance y fuentes

1.1 Propósito

El presente SAD define la arquitectura técnica de HELA y establece la organización de sus principales elementos de software. Su finalidad es proporcionar una referencia coherente para la construcción del sistema, describiendo los límites, contenedores, componentes, reglas de persistencia, controles de seguridad, integraciones y decisiones arquitectónicas necesarias.

La descripción se concentra en las decisiones que afectan la estructura del sistema. Los detalles de pantallas, diseño físico de base de datos y código específico se mantienen fuera de este documento cuando corresponden al diseño detallado o a la implementación.

1.2 Alcance

La arquitectura cubre la carta QR pública, el panel interno del restaurante, la administración SaaS, la API REST, la persistencia relacional, el control de acceso, la auditoría, la facturación electrónica y la impresión. Estos elementos soportan los requisitos, reglas de negocio, casos de uso CU-01 a CU-29 y procesos principales definidos para HELA.

Dentro del sistema. Se consideran el frontend web, la API REST, los módulos funcionales, la capa sencilla de acceso a datos y la base MySQL.

Fuera del sistema. La preparación física de alimentos y bebidas, el procesamiento realizado por el proveedor de facturación electrónica y el procesamiento físico de impresión permanecen fuera del límite de HELA.

1.3 Fuentes documentales

Tabla 3

Fuentes documentales de la arquitectura

Fuente	Uso dentro del SAD
HELA - Especificación del Sistema v1.0	Fuente funcional principal para actores, módulos, requisitos, reglas de negocio, casos de uso, procesos, estados y criterios de aceptación.
HELA - Análisis del Sistema v2.1	Fuente de contraste del alcance funcional y trazabilidad heredada.
hela-openapi.yaml · HELA API 0.2.0	Fuente de verdad del contrato HTTP: rutas, parámetros, solicitudes, respuestas, errores y x-hela-cu.
IS1-26 S02 - Guía de Referencia Rápida	Establece OpenAPI como fuente de verdad y la necesidad de mantener coherencia entre capas y contrato.
ISO/IEC/IEEE 42010:2022	Marco de referencia para organizar una descripción de arquitectura.
C4 Model	Convención utilizada para las vistas de contexto, contenedores y componentes.

2. Drivers arquitectónicos

Los drivers arquitectónicos son condiciones del sistema que influyen directamente en su organización técnica. Se derivan del alcance funcional y de los atributos de calidad que HELA debe conservar.

2.1 Drivers funcionales

SaaS multi-restaurante y multi-sucursal. Las operaciones internas deben mantener el contexto del restaurante y, cuando corresponda, de la sucursal.

Carta QR pública. El cliente final accede a capacidades públicas sin obtener los privilegios de los usuarios internos.

Operación de pedidos. Los pedidos oficiales originan comandas y mantienen relación con atención, precuenta, pagos y documentos.

Caja y pagos. Los saldos, pagos parciales o mixtos, sesiones de caja y anulaciones requieren consistencia.

Inventario. Las existencias y movimientos se controlan por sucursal.

Administración SaaS. Las funciones del Super Admin se mantienen separadas de la operación cotidiana de un restaurante.

Integraciones. Facturación electrónica e impresión deben conservar el estado interno de la operación aun cuando el servicio externo falle.

2.2 Drivers de calidad

Tabla 4

Drivers arquitectónicos

Driver	Implicancia arquitectónica
Seguridad	Autenticación para canales internos, autorización en backend y validación del ámbito de restaurante/sucursal.
Separación de información	Las consultas y operaciones se ejecutan dentro del contexto autorizado del usuario.
Consistencia	Pedidos, pagos, saldos, caja, documentos y anulaciones utilizan transacciones cuando una operación afecta varios registros relacionados.
Trazabilidad	Las acciones sensibles conservan responsable, fecha, contexto, acción y resultado.
Mantenibilidad	La solución se organiza por módulos funcionales con responsabilidades identificables y estructura interna sencilla.
Usabilidad	El frontend permite interfaces diferenciadas para cliente, operación interna y administración, manteniendo especial atención a la carta QR móvil.
Continuidad ante fallos externos	Facturación e impresión no eliminan la operación interna cuando una dependencia externa falla.

2.3 Restricciones arquitectónicas

Sistema web. La interacción principal se realiza desde un navegador mediante HTML, CSS y JavaScript.

Contrato HTTP. La API debe respetar el contrato OpenAPI 3.0.3 vigente y no incorporar rutas o formatos no declarados.

Monolito modular. El backend se construye como una sola aplicación Node.js/Express organizada por módulos funcionales; HELA no se divide en microservicios.

Persistencia relacional. La línea base utiliza MySQL y acceso desde Node.js mediante mysql2 o un driver compatible.

Entorno inicial. La construcción y validación se realizan inicialmente en un entorno local. La publicación externa se define posteriormente.

3. Arquitectura de solución

3.1 Estilo arquitectónico

HELA adopta un monolito modular expuesto mediante una API REST. El backend se ejecuta como una única aplicación Node.js con Express.js, mientras que sus responsabilidades se separan por módulos funcionales. Esta organización permite mantener un sistema sencillo de desplegar y comprender, sin perder separación entre autenticación, catálogo, pedidos, caja, inventario, administración y demás áreas del dominio.

La API recibe las solicitudes HTTP, aplica autenticación y autorización cuando corresponde, delega la operación a los servicios del módulo y utiliza repositorios para acceder a MySQL. Las integraciones externas se invocan desde servicios específicos cuando una operación funcional lo requiere.

3.2 Principios de diseño

Responsabilidad clara. Cada módulo concentra funciones relacionadas y evita asumir responsabilidades de otros módulos sin una necesidad concreta.

Estructura sencilla. Rutas, controladores, servicios y repositorios se utilizan cuando aportan una responsabilidad identificable; no se incorporan capas adicionales por convención.

Contrato primero. OpenAPI define la interfaz HTTP que debe respetar la implementación.

Separación entre frontend y backend. El frontend presenta la información y solicita operaciones; las reglas de negocio y la autorización se mantienen en el backend.

Persistencia controlada. El acceso a MySQL se concentra en repositorios o funciones de acceso a datos y utiliza transacciones cuando corresponde.

Integraciones desacopladas. Facturación e impresión no determinan el estado interno de la operación comercial; sus resultados se registran y controlan.

3.3 Línea base tecnológica

Tabla 5

Línea base tecnológica

Área	Tecnología	Responsabilidad
Frontend	HTML5 + CSS3 + JavaScript Vanilla	Interfaces de carta QR, panel interno y administración SaaS. Comunicación con la API mediante Fetch API.
Backend	Node.js + Express.js + JavaScript	API REST, rutas, controladores, servicios, autorización, reglas e integraciones.
Persistencia	MySQL	Información SaaS, operativa, comercial y de auditoría.
Acceso a datos	mysql2 o driver compatible	Consultas parametrizadas, acceso relacional y transacciones desde Node.js.
Contrato	OpenAPI 3.0.3	Definición versionada de rutas, parámetros, solicitudes, respuestas, errores y trazabilidad.
Autenticación	Bearer JWT conforme a OpenAPI	Sesiones de usuarios internos y administración SaaS.

Área	Tecnología	Responsabilidad
Entorno inicial	Servidor local de desarrollo y validación	Frontend, API y base de datos disponibles en el entorno local; API declarada en localhost:3000.

3.4 Flujo general de la solución

El flujo técnico general mantiene una secuencia directa entre el navegador, la API y la persistencia. El frontend ejecuta HTML, CSS y JavaScript en el navegador y utiliza Fetch API para enviar y recibir información JSON. Express recibe las solicitudes, aplica los controles necesarios, ejecuta el servicio del módulo y accede a MySQL mediante el repositorio correspondiente.



4. Vistas arquitectónicas C4

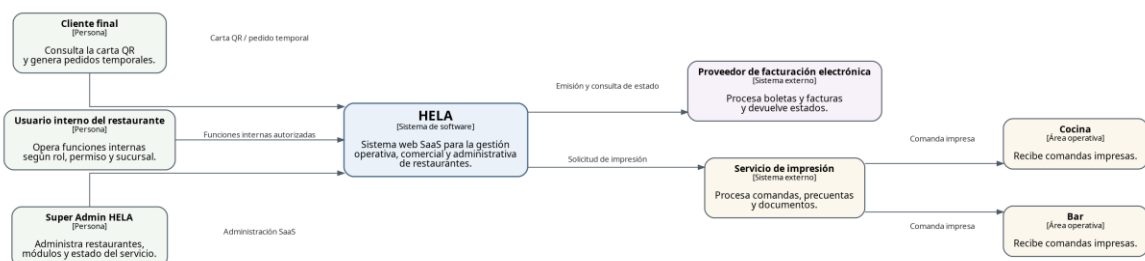
Las vistas C4 presentan la arquitectura desde tres niveles complementarios. La vista de contexto muestra HELA frente a personas y sistemas externos; la vista de contenedores identifica las principales unidades de software; y la vista de componentes describe la organización interna de la API.

4.1 C4 Nivel 1 - Contexto

La vista de contexto delimita HELA frente a sus usuarios y dependencias externas. El Cliente final utiliza la carta QR; los usuarios internos operan las funciones del restaurante; y el Super Admin gestiona funciones SaaS. Facturación electrónica e impresión permanecen como sistemas externos. Cocina y Bar reciben las comandas como áreas operativas.

Figura 1

C4 Nivel 1 - Contexto de HELA



Nota. Elaboración propia con base en la Especificación del Sistema y el alcance funcional vigente de HELA.

4.2 C4 Nivel 2 - Contenedores

La vista de contenedores identifica tres elementos principales dentro de HELA: el frontend web, la API REST y la base MySQL. El frontend contiene la experiencia de usuario y consume la API mediante HTTP/JSON. La API implementa las operaciones del contrato, aplica reglas y accede

a la persistencia. MySQL conserva la información del sistema. Facturación electrónica e impresión se representan fuera del límite de HELA.

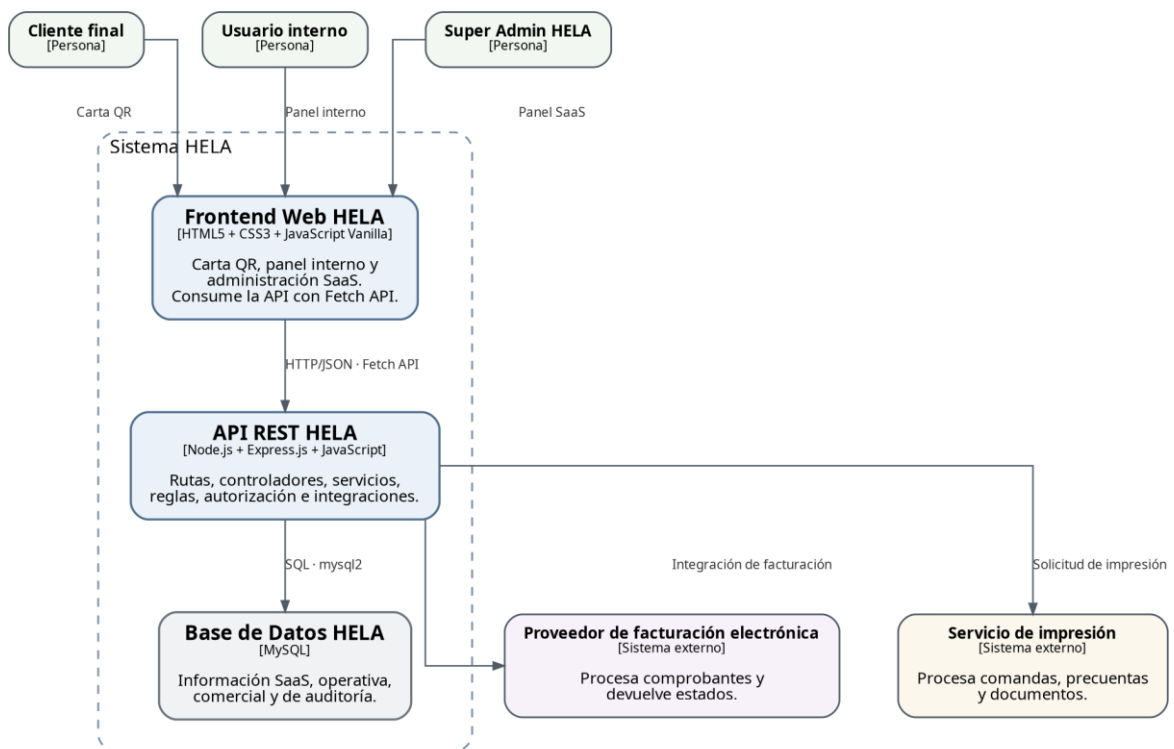
Tabla 6

Responsabilidades de los contenedores

Contenedor	Tecnología	Responsabilidad
Frontend Web HELA	HTML5, CSS3, JavaScript Vanilla	Presenta la carta QR, panel interno y administración SaaS. Consume la API con Fetch API.
API REST HELA	Node.js, Express.js, JavaScript	Expone las operaciones HTTP, aplica autorización, ejecuta servicios y coordina integraciones.
Base de Datos HELA	MySQL	Conserva información relacional del sistema y permite operaciones transaccionales.

Figura 2

C4 Nivel 2 - Contenedores de HELA



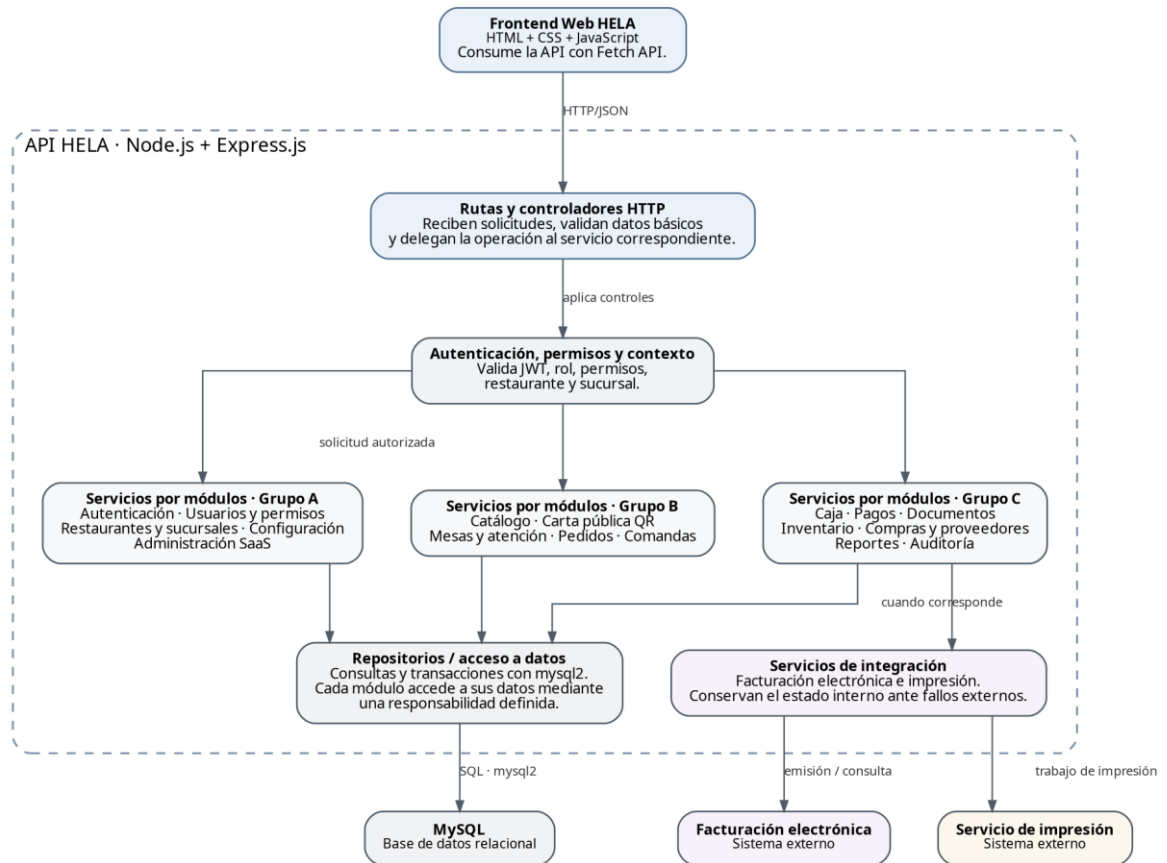
Nota. La separación mostrada es lógica. En el entorno local, el servidor Express puede entregar los archivos estáticos del frontend y exponer la API bajo el mismo origen.

4.3 C4 Nivel 3 - Componentes de la API HELA

La API se organiza alrededor de rutas y controladores HTTP, controles de autenticación y contexto, servicios agrupados por módulos, repositorios de acceso a datos y servicios de integración. Esta vista representa responsabilidades arquitectónicas y no obliga a crear capas adicionales fuera de las necesarias para cada módulo.

Figura 3

C4 Nivel 3 - Componentes de la API HELA



Nota. Los módulos se agrupan para mantener legibilidad. En el código cada módulo puede disponer de rutas, controlador, servicio y repositorio según su responsabilidad.

5. Organización del código

5.1 Estructura general

La organización del código mantiene separados el frontend y el backend dentro de src. Los nombres de carpetas se expresan principalmente en español y la estructura busca que un estudiante pueda ubicar con facilidad dónde se encuentra cada responsabilidad.

```

src/
├── frontend/
│   ├── paginas/
│   ├── estilos/
│   ├── scripts/
│   ├── componentes/
│   ├── modulos/
│   └── imagenes/
└── backend/
    ├── servidor.js
    ├── configuracion/
    ├── base-datos/
    │   └── conexion.js
    ├── modulos/
    │   ├── autenticacion/
    │   ├── usuarios-permisos/
    │   └── restaurantes-sucursales/
  
```

```

|— catalogo/
|— carta-publica/
|— mesas-atencion/
|— pedidos/
|— comandas/
|— caja/
|— pagos/
|— documentos/
|— inventario/
|— compras-proveedores/
|— reportes/
|— auditoria/
|— configuracion/
|— administracion-saas/

```

5.2 Organización de un módulo backend

Cada módulo puede utilizar una estructura interna sencilla. Los archivos se incorporan cuando cumplen una responsabilidad concreta; un módulo pequeño no necesita crear archivos vacíos únicamente para repetir una plantilla.

```

modulo/
|— rutas.js
|— controlador.js
|— servicio.js
|— repositorio.js

```

rutas.js. Declara las rutas Express del módulo y las relaciona con el controlador correspondiente.

controlador.js. Recibe la solicitud HTTP, obtiene los datos necesarios y devuelve la respuesta definida por el contrato.

servicio.js. Aplica las reglas y coordina la operación funcional del módulo.

repositorio.js. Realiza consultas y actualizaciones sobre MySQL mediante el driver de acceso a datos.

5.3 Criterios de organización

Sin duplicación de reglas. Las reglas de negocio se aplican en el backend y no se duplican como autoridad funcional en el frontend.

Sin rutas inventadas. Los archivos de rutas se construyen a partir de `hela-openapi.yaml`.

Dependencias claras. Un módulo utiliza otro servicio solamente cuando existe una relación funcional necesaria.

Simplicidad. No se crean patrones, carpetas o abstracciones que no resuelvan una responsabilidad identificable del sistema.

6. Persistencia y aislamiento de información

6.1 Persistencia relacional

HELA utiliza MySQL como base de datos relacional. La persistencia conserva la información de restaurantes, sucursales, usuarios, catálogo, mesas, pedidos, comandas, caja, pagos, documentos, inventario, compras, auditoría y administración SaaS. El diseño físico de tablas, claves, índices y restricciones se documenta en el Diseño Detallado del Sistema.

6.2 Acceso a datos

El backend accede a MySQL mediante mysql2 o un driver compatible. Las consultas deben utilizar parámetros y evitar la construcción de sentencias SQL mediante concatenación directa de datos proporcionados por el usuario. Las conexiones se centralizan en la configuración de base de datos para evitar credenciales repetidas dentro de los módulos.

6.3 Aislamiento por restaurante y sucursal

La separación de información se mantiene como una regla transversal. Las operaciones internas resuelven el restaurante autorizado a partir del contexto de autenticación y limitan la sucursal cuando la función depende de una sede específica. Los identificadores enviados por el cliente no sustituyen la validación del ámbito permitido.

Tabla 7

Reglas de persistencia y aislamiento

Regla	Aplicación
Aislamiento por restaurante	Toda consulta interna se limita al restaurante autorizado para el usuario o contexto SaaS correspondiente.
Aislamiento por sucursal	Mesas, caja, inventario y demás operaciones por sede se limitan a las sucursales autorizadas.
Integridad histórica	Pedidos, pagos, documentos, anulaciones y auditoría conservan información suficiente para reconstruir la operación.
Transacciones	Las operaciones que modifican varios registros relacionados se ejecutan de forma transaccional cuando la consistencia lo requiere.
Inventario	El stock se controla por sucursal mediante movimientos registrados.

6.4 Transacciones e integridad histórica

Las operaciones relacionadas con pedido oficial, pagos, cierre de caja, documentos y anulaciones pueden afectar varios registros. En esos casos el servicio inicia una transacción, ejecuta las operaciones necesarias mediante los repositorios y confirma los cambios únicamente cuando la operación completa puede mantenerse consistente. Si se produce un error, la transacción se revierte.

Los registros históricos y de auditoría no se eliminan libremente cuando son necesarios para explicar una operación anterior. Las reglas de conservación se mantienen coherentes con RN-HIS-01, RN-AUD-01 y RN-ANU-01 de la Especificación del Sistema.

7. Seguridad

7.1 Autenticación

El contrato OpenAPI define autenticación HTTP Bearer con formato JWT para las operaciones internas. El flujo contractual incluye inicio de sesión, renovación y cierre de sesión. La API Express valida el token antes de ejecutar las funciones protegidas y mantiene separadas las capacidades públicas de la carta QR.

7.2 Autorización y contexto

La autorización se realiza en el backend. Antes de ejecutar una operación sensible se comprueban los permisos, el restaurante, la sucursal cuando corresponda, los módulos habilitados y

el estado de la entidad afectada. El hecho de ocultar una opción en el frontend no sustituye esta validación.

7.3 Canal público QR

La carta QR pública no utiliza los privilegios de una sesión interna. Las operaciones públicas se limitan al contexto permitido por el QR y a los endpoints declarados para Carta QR pública. Un pedido temporal no produce los efectos de un pedido oficial hasta cumplir la validación funcional correspondiente.

7.4 Credenciales y consultas

Las contraseñas se almacenan mediante un mecanismo de hash seguro y no se conservan en texto legible. Las credenciales de base de datos y de servicios externos se mantienen fuera del código fuente. Las consultas SQL utilizan parámetros para reducir el riesgo de inyección y mantener una separación clara entre datos y sentencias.

7.5 Auditoría

Tabla 8

Controles de seguridad

Control	Criterio
Autenticación	Obligatoria para funciones internas y SaaS; la carta QR expone únicamente capacidades públicas.
Autorización	El backend valida permiso, módulo habilitado, restaurante, sucursal y estado de la operación.
JWT	Las operaciones protegidas utilizan Bearer JWT conforme al esquema declarado en OpenAPI.
Credenciales	Contraseñas con hash; secretos y credenciales fuera del código fuente.
SQL	Consultas parametrizadas mediante mysql2 o mecanismo equivalente del driver seleccionado.
Auditoría	Anulaciones, descuentos, pagos, caja, permisos, inventario y soporte conservan trazabilidad.

8. Integraciones externas

8.1 Facturación electrónica

HELA mantiene la integración con un proveedor externo de facturación electrónica para las operaciones de boleta y factura. El backend conserva la solicitud y el estado interno relacionado con la operación comercial. Un error del proveedor no elimina el pedido, el pago ni el documento interno que originó la solicitud.

8.2 Impresión

La impresión se utiliza para comandas, precuentas y documentos cuando corresponda. La operación original permanece registrada aunque la impresión falle. Una reimpresión se trata como una nueva solicitud asociada al mismo elemento funcional, conservando trazabilidad.

8.3 Archivos y recursos

La línea base no define un servicio externo independiente de almacenamiento. Los recursos estáticos del frontend, como imágenes utilizadas por la interfaz, forman parte del proyecto. Si una necesidad funcional requiere almacenamiento de archivos cargados por usuarios, el mecanismo se definirá en el Diseño Detallado y se incorporará al SAD únicamente cuando exista una decisión confirmada.

8.4 Tratamiento de fallos

Tabla 9

Integraciones externas

Integración	Comportamiento
Facturación electrónica	Un fallo externo conserva la operación comercial interna y registra un estado que permita conocer el resultado o realizar una acción posterior.
Impresión	Un fallo de impresión no elimina comanda, precuenta ni documento; la solicitud puede volver a ejecutarse de forma controlada.

9. Despliegue

9.1 Entorno local inicial

El entorno inicial de desarrollo y validación utiliza un servidor local. Node.js ejecuta la aplicación Express, la cual expone la API REST en `http://localhost:3000`, dirección declarada actualmente en `hela-openapi.yaml`. Para simplificar el entorno, Express puede servir también los archivos estáticos del frontend desde el mismo proyecto.

9.2 Distribución local

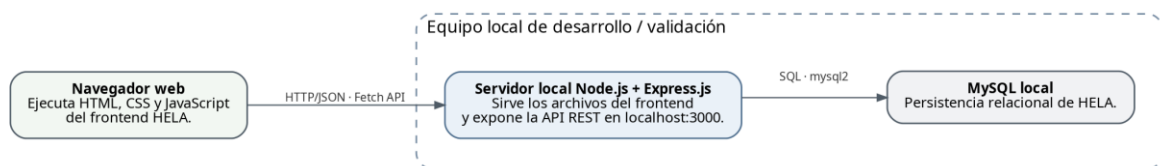
Tabla 10

Componentes del despliegue local

Elemento	Ubicación inicial	Función
Navegador web	Equipo del usuario o desarrollador	Ejecuta HTML, CSS y JavaScript del frontend.
Servidor Node.js + Express	Equipo local de desarrollo/validación	Sirve el frontend y expone la API REST.
MySQL	Equipo local de desarrollo/validación	Conserva la información relacional del sistema.

Figura 4

Despliegue local inicial de HELA



Nota. El despliegue representa el entorno inicial de construcción y validación. La publicación externa se define como una decisión posterior.

9.3 Publicación externa

La infraestructura externa de publicación no forma parte de la línea base actual. Su selección deberá considerar facilidad de administración, costo, capacidad requerida, respaldo y acceso seguro. Esta decisión no modifica el contrato HTTP ni la separación lógica entre frontend, API y persistencia.

10. Flujos arquitectónicos representativos

Los siguientes flujos describen cómo colaboran los elementos principales de la arquitectura en operaciones representativas. No sustituyen los casos de uso ni los procesos funcionales; muestran el recorrido técnico de la solicitud.

Tabla 11

Flujos arquitectónicos representativos

Flujo	Secuencia técnica resumida
Inicio de sesión	Frontend → POST /api/v1/autenticacion/sesiones → controlador → servicio de autenticación → repositorio → MySQL → respuesta con sesión/token.
Pedido temporal QR	Frontend público → endpoint OpenAPI de carta/pedido temporal → servicio del módulo → repositorio → MySQL → referencia y estado temporal.
Validación y pedido oficial	Panel interno → endpoint de validación → autenticación/contexto → servicio de pedidos → transacción → MySQL → pedido oficial.
Comanda	Pedido oficial → servicio de comandas → persistencia → solicitud de impresión → servicio externo; el pedido/comanda permanece registrado ante error externo.
Pago y documento	Panel interno → servicio de pagos → transacción → MySQL → actualización de saldo → servicio de documentos cuando corresponde.
Facturación electrónica	Documento → servicio de integración → proveedor externo → registro del estado devuelto sin perder la operación interna.

10.1 Inicio de sesión

El frontend envía las credenciales mediante el endpoint contractual de inicio de sesión. El controlador delega la validación al servicio de autenticación, el cual consulta la información necesaria y genera la respuesta definida por OpenAPI. Las operaciones posteriores protegidas utilizan el token Bearer JWT.

10.2 Pedido temporal QR y validación

El cliente utiliza el frontend público para consultar la carta y generar un pedido temporal. El backend registra el pedido sin producir efectos de pedido oficial. Posteriormente, un usuario interno autorizado ejecuta la validación; el servicio comprueba el estado y crea el pedido oficial dentro de una operación consistente.

10.3 Pedido oficial y comanda

Un pedido oficial puede generar comandas de acuerdo con el destino de preparación de los productos. El servicio registra la comanda antes de solicitar la impresión correspondiente. Si la impresión falla, la comanda permanece disponible para consulta o reimpresión controlada.

10.4 Pago y documento comercial

El registro de un pago actualiza el saldo de la atención y mantiene relación con la caja, usuario y sucursal cuando corresponde. La emisión del documento comercial se realiza después de validar las condiciones funcionales y conserva una relación independiente con los pagos aplicados.

10.5 Facturación electrónica

Cuando el documento requiere facturación electrónica, el backend prepara la solicitud para el proveedor externo y registra el estado correspondiente. La respuesta del proveedor se conserva como parte de la trazabilidad sin convertir al servicio externo en autoridad sobre la operación interna de HELA.

11. Contrato OpenAPI

11.1 Fuente de verdad HTTP

El archivo `hela-openapi.yaml` es la fuente de verdad del contrato HTTP de HELA. Define las rutas, métodos, parámetros, cuerpos de solicitud, respuestas, errores, esquemas de datos y relaciones con casos de uso mediante la extensión `x-hela-cu`. La implementación Express debe ajustarse a este contrato.

11.2 Independencia tecnológica

OpenAPI describe el comportamiento HTTP sin depender de Node.js, Express.js, MySQL ni de la estructura interna de carpetas. Por este motivo, el cambio de tecnología no modifica por sí mismo las rutas o formatos contractuales. Un cambio de endpoint requiere una decisión funcional o contractual independiente.

11.3 Cobertura contractual vigente

El contrato vigente declara OpenAPI 3.0.3, título HELA API y versión 0.2.0. Contiene 61 rutas, 76 operaciones y 89 esquemas. Las operaciones cubren los casos de uso CU-01 a CU-29 mediante `x-hela-cu`. El servidor de desarrollo declarado es `http://localhost:3000`.

11.4 Reglas de implementación

Rutas. Express debe declarar únicamente rutas compatibles con el contrato vigente.

Solicitudes y respuestas. Los nombres, formatos y campos deben respetar los esquemas OpenAPI correspondientes.

Errores. Las respuestas de error se mantienen dentro de los estados y estructuras documentadas por el contrato.

Trazabilidad. Cada operación implementada conserva relación con los casos de uso indicados mediante `x-hela-cu`.

12. Trazabilidad arquitectónica

12.1 Módulos y componentes

Tabla 12

Trazabilidad módulo-componente

Módulo funcional	Componentes arquitectónicos principales
MF-01 Acceso, usuarios y permisos	Autenticación, permisos y contexto; servicios de autenticación/usuarios; repositorios

Módulo funcional	Componentes arquitectónicos principales
MF-02 Administración SaaS	Servicios de administración SaaS; contexto y autorización
MF-03 Restaurante y sucursales	Servicios de restaurantes/sucursales; repositorios
MF-04 Productos y carta QR	Servicios de catálogo y carta pública; frontend QR
MF-05 Mesas y atención	Servicios de mesas y atención; repositorios
MF-06 Pedidos	Servicios de pedidos; transacciones y repositorios
MF-07 Comandas	Servicios de comandas; integración de impresión
MF-08 Caja y pagos	Servicios de caja y pagos; transacciones y repositorios
MF-09 Documentos y facturación	Servicios de documentos; integración de facturación electrónica
MF-10 Inventario, compras y proveedores	Servicios de inventario/compras; repositorios
MF-11 Reportes y auditoría	Servicios de reportes y auditoría
MF-12 Configuración	Servicios de configuración y autorización

12.2 Casos de uso y contrato

Tabla 13

Trazabilidad OpenAPI por etiqueta

Etiqueta OpenAPI	Casos de uso relacionados
Autenticación	CU-01
Usuarios y permisos	CU-02
Restaurante y sucursales	CU-03
Catálogo	CU-04
Carta QR pública	CU-05, CU-06
Mesas y atención	CU-07, CU-13, CU-15, CU-16
Pedidos	CU-08, CU-09, CU-10, CU-12, CU-17, CU-29
Comandas	CU-11, CU-22
Caja	CU-14
Pagos	CU-15, CU-16, CU-18
Documentos	CU-19, CU-20, CU-21, CU-22
Inventario	CU-23
Compras y proveedores	CU-24
Reportes	CU-25
Auditoría	CU-26
Configuración	CU-27
Administración SaaS	CU-28
Impresión	CU-11, CU-15, CU-20, CU-22

12.3 Cadena de trazabilidad

La arquitectura mantiene una cadena de trazabilidad que permite recorrer la necesidad funcional hasta la implementación y la verificación. La Especificación define el requisito, la regla y el caso de uso; OpenAPI define el contrato HTTP; el SAD asigna la responsabilidad arquitectónica; y el código implementa la operación dentro del módulo correspondiente.

Requisito / regla de negocio
 Caso de uso CU-XX
 Proceso funcional DP-XX
 Operación hela-openapi.yaml
 Ruta / controlador / servicio / repositorio
 Prueba

13. Escenarios de calidad

Los escenarios de calidad expresan situaciones verificables que ayudan a evaluar si la arquitectura responde a los atributos definidos para HELA. No reemplazan los requisitos no funcionales; los relacionan con decisiones concretas de arquitectura.

Tabla 14

Escenarios de calidad

Atributo	Escenario	Respuesta arquitectónica
Seguridad	Un usuario intenta ejecutar directamente una operación para la cual no tiene permiso.	La API valida JWT, permisos y contexto antes de invocar el servicio; la solicitud se rechaza.
Separación de información	Un usuario de un restaurante intenta consultar información de otro restaurante.	El contexto autorizado limita la consulta antes del acceso a MySQL.
Consistencia	Un pago afecta saldo, caja y registros asociados y ocurre un error durante la operación.	El servicio utiliza una transacción y revierte el conjunto de cambios si no puede completarse.
Continuidad	El proveedor de facturación o impresión no responde.	La operación interna permanece registrada y la integración conserva un estado controlado.
Mantenibilidad	Debe modificarse una función de pedidos sin alterar reportes o inventario.	La lógica se localiza dentro del módulo y sus dependencias explícitas, reduciendo efectos innecesarios.
Trazabilidad	Se requiere conocer quién anuló una operación sensible.	Auditoría conserva responsable, fecha, contexto, acción y resultado.

14. Riesgos y decisiones pendientes

14.1 Riesgos principales

Acceso indebido entre restaurantes. Una consulta sin contexto adecuado podría exponer información fuera del ámbito autorizado; por ello el aislamiento se aplica antes del acceso a datos.

Inconsistencia transaccional. Operaciones como pagos, documentos o anulaciones pueden dejar datos parciales si no se gestionan como una unidad cuando corresponde.

Dependencia de servicios externos. Facturación e impresión pueden fallar o responder con demora; el sistema debe conservar la operación interna y un estado controlado.

Divergencia entre código y contrato. Una ruta o formato implementado fuera de OpenAPI rompe la fuente de verdad y dificulta pruebas y trazabilidad.

Crecimiento desordenado del backend. Agregar capas, carpetas o dependencias sin responsabilidad concreta puede dificultar el mantenimiento; la estructura modular debe mantenerse simple.

14.2 Decisiones pendientes

Tabla 15

Decisiones pendientes

ID	Tema	Estado documental
DA-01	Proveedor concreto de facturación electrónica	Pendiente de selección; la integración mantiene independencia del proveedor específico.
DA-02	Mecanismo concreto de impresión local	Pendiente de selección; HELA conserva la solicitud y la trazabilidad de impresión.
DA-03	Vigencia exacta del pedido temporal QR	Pendiente de definición funcional y contractual.
DA-04	Duración de tokens de acceso y renovación	Pendiente de cierre en la política técnica de seguridad.
DA-05	Servidor o proveedor de publicación externa	Pendiente; el entorno vigente de desarrollo y validación es local.

15. Artefactos vinculados

15.1 Ubicación oficial

Tabla 16

Registro oficial de artefactos

Artefacto	Ubicación oficial	Referencia
SAD oficial	docs/arquitectura/HELA_SAD_v2.0.pdf	Documento principal de arquitectura.
SAD editable	docs/arquitectura/HELA_SAD_v2.0.docx	Fuente editable de la versión vigente.
C4 N1	docs/arquitectura/diagramas/HELA_C4_N1_Contexto_v2.0.png	Figura 1.
C4 N2	docs/arquitectura/diagramas/HELA_C4_N2_Contenedores_v2.0.png	Figura 2.
C4 N3	docs/arquitectura/diagramas/HELA_C4_N3_Componentes_API_v2.0.png	Figura 3.
Despliegue local	docs/arquitectura/diagramas/HELA_Despliegue_Local_v2.0.png	Figura 4.
Contrato OpenAPI	docs/contratos/hela-openapi.yaml	Fuente de verdad del contrato HTTP; nombre estable.
Especificación del Sistema	docs/arquitectura/HELA_Especificacion_del_Sistema_v1.0.pdf	Fuente funcional principal.
Código	src/	Implementación del producto.

15.2 Regla de referencia

Cada artefacto se cita por nombre y ubicación dentro del repositorio oficial. El contrato OpenAPI conserva un nombre estable y su versión se mantiene dentro de info.version y del historial del repositorio. Las versiones anteriores del SAD y de sus diagramas permanecen como antecedentes documentales y no se sobrescriben.

16. Referencias